

CSC 453

Spring 2026 - Day 17

Admin

- Lab due Tonight
- Quiz due next Monday
- I will hold office hours on Monday from 12-3pm
- Reminder the final is on Tuesday, June 9th at 13:10 in the lab room

Disk scheduling

Questions to consider

- Why does the OS (or disk controller) bother reordering I/O requests rather than just serving them in arrival order?
- What problem does SSTF solve compared to FCFS, and what new problem does it introduce?
- How does SCAN (elevator) avoid the starvation problem of SSTF, and why is it unfair to requests that just arrived behind the head?
- What is the difference between SCAN and LOOK, and why is LOOK generally preferred?

Disk scheduling

- We know that disk performance is punitive; seek dominates
- OSes typically have many processes issuing reads/writes; I/O is bursty
 - queues can grow large, then drain
- This gives us an opportunity to reorder requests to reduce costly seeks
- Note: these algos used to run in the OS; drives have their own controllers now

Disk scheduling: FCFS and SSTF

- **FCFS**: serve requests in arrival order
 - Fair, no starvation — but makes no attempt to minimize seek time
- **SSTF**: always service the request closest to the current head position
 - Tends to minimize average seek time
 - Con: starvation — a request far from the head may never be serviced if closer requests keep arriving

Disk scheduling: SCAN family

- **SCAN** (elevator): sweep in one direction servicing requests, reverse at the end
 - Solves starvation, but unfair — requests just behind the head wait a full sweep
- **C-SCAN**: like SCAN but jump back to the start instead of reversing; more uniform wait times
- **LOOK** / **C-LOOK**: like SCAN/C-SCAN but turn around at the last actual request, not the disk edge
 - Less wasted movement; usually what people mean when they say “SCAN”

Disk scheduling example

- Disk has 200 cylinders; head is at cylinder 50; pending requests:
 - 82, 170, 43, 140, 24, 16, 190
- What is the total head movement for FCFS, SSTF, and SCAN?

Disk scheduling summary

- Which should we choose?
- Generally speaking, SCAN family is used
 - Good for heavy disk loads, avoid starvation
 - What if you had a system where the average queue length was 1?
- Linux has a **deadline scheduler**
 - Separate read and write queues (prioritize reads as processes are more likely to block on read vs write)
 - Ordered queues: so they end up being serviced in a SCAN-like order
 - Also FCFS queues for both for bookkeeping
 - After a batch of actions, check to see if there are any in the FCFS queue that has aged beyond some threshold

What isn't clear?

Comments? Thoughts?

Storage organization

Questions to consider

- Why is a single disk insufficient for many modern applications, and what two software/hardware abstractions help organize multiple disks?
- What is a RAID stripe, and how does stripe size affect throughput and internal fragmentation?
- How does RAID 0 differ from RAID 1 in terms of performance and redundancy?
- How does XOR parity allow RAID 4/5 to reconstruct data after a single disk failure?
- What is the “small write problem” in RAID 5, and why does it occur?

Storage organization

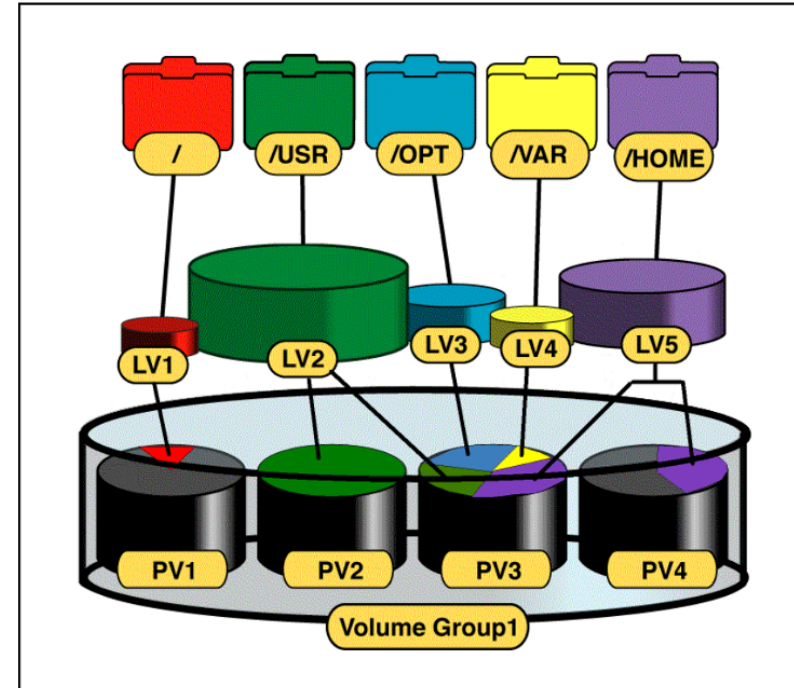
- Single disks neither have the capacity, responsiveness, throughput or reliability to be supportive of many applications we desire
 - High-bandwidth applications: multi-media, scientific computing
 - High-capacity: “big data” applications
 - High-availability: any online service
- We’re limited by physics to increase capacity: bit-densities are approaching theoretical limits
- Form factor and energy limit the physical size

Storage organization (cont'd)

- What should we do?
 - Add more disks
- Q: How do we organize multiple drives into a single logical unit of storage?
 - LVM
 - RAID

LVM

- Logical Volume Manager: a way of organizing/abstracting multiple block devices into a single logical unit of storage
- A **software** translation layer
- Can provide additional features, like: encryption, snapshots, redundancy, hot-swapping, resizing



RAID

- **Redundant Array of Independent Disks**: another type of volume manager
- Often a physical organization of disks, and a hardware controller, providing a single logical view of all disks
 - (Although software controllers exist)
- Can provide: high throughput, parallel I/O, reliability and transparent management

RAID (cont'd)

- Consider Mean-time-to-failure (MTTF)
 - e.g., 750,000 hours for a single disk
 - A data center has 1,000,000 disks
 - 1 disk failure every .75 hours (45 minutes)
- RAID gives us a way to tolerate and recover from these failures gracefully

RAID terminology

- **Stripe**: a set of blocks across multiple disks that are accessed together
 - How does stripe size impact system characteristics?
 - Smaller means more flexibility in size of data stored (reduced internal fragmentation); decreased maximum throughput
 - Larger means high throughput; higher internal fragmentation
- **Mirroring**: storing identical data on multiple disks for redundancy
 - Reads are faster (can read from either disk), writes are slower (need to write to both disks)
 - Recovering from a failure is effectively instant: just read from the surviving disk
- **Parity**: storing information that can be used to reconstruct data in case of a disk failure

RAID levels - RAID 0

- **RAID 0**: striping without redundancy
- logically, it just looks like one large disk
 - e.g., Two 1TB drives becomes one 2TB drive
- Pros?
 - **High Performance**: Data is split across multiple disks, allowing for faster read and write speeds.
 - **Full Capacity Utilization**: All disk space is used for storage, with no redundancy overhead.
- Cons?
 - **No Redundancy**: If one disk fails, all data is lost.

RAID levels - RAID 1

- **RAID 1**: mirroring
- Two identical RAID sets (mirror copy of each disk)
- Individual drives within a RAID set can fail, so long as their logical counterpart survives
- Reads can be serviced by either set (e.g. whichever responds first)
- Pros?
 - **High Redundancy**: Data is mirrored across two disks, providing a complete backup.
 - **Improved Read Performance**: Read operations can be faster as data can be read from either disk.
- Cons?
 - **Storage Efficiency**: Only half of the total disk space is usable, as the other half is used for mirroring.

RAID levels - RAID 2-4

- Designate a disk as a parity disk; parity being encoded data for redundancy
 - **RAID 2**: bit-level striping with dedicated Hamming code parity
 - **RAID 3**: byte-level striping with dedicated parity disk
 - **RAID 4**: block-level striping with dedicated parity disk
- Many different encoding schemes; here's a simple one:
 - $A \oplus B \oplus C = P$
 - A dies; $P \oplus B \oplus C = A$
 - All have a *write bottleneck*: only as fast as the parity drive

RAID 4

- Pros?
 - **Good Read Performance**: Similar to RAID 0, data is striped across disks, improving read speeds.
 - **Parity for Redundancy**: A dedicated parity disk provides data protection in case of a single disk failure. But the overhead of parity is only 1 disk, so more efficient than RAID 1
- Cons?
 - **Write Bottleneck**: The dedicated parity disk can become a bottleneck during write operations.
 - **Reduced Fault Tolerance**: If the parity disk fails, the array loses all redundancy — any subsequent disk failure will cause data loss.

RAID levels - RAID 5

- **RAID 5**: block-level striping with distributed parity
- Like RAID 4, but with parity distributed among all disks
- only supports a single drive failure, but ...
- ... All drives perform in reads and writes
- Best small read, large read, large write performance
 - TERRIBLE small write performance
 - **Small write problem**: small-writes incur a lot of I/O; e.g., a small update to a large stripe
 - Entire stripe read in, modified, new parity computed, written back out

RAID 5

- Pros?
 - **Balanced Performance**: Offers good read and write performance with data and parity distributed across all disks.
 - Though this depends on the workload: small writes are very bad, large writes are good
 - **Efficient Storage**: Only one disk's worth of space is used for parity, providing better storage efficiency than RAID 1.
- Cons?
 - **Rebuild Time**: Rebuilding the array after a disk failure can be time-consuming and impacts performance.
 - **Write Penalty**: Write operations (small writes are the worst) are slower due to the need to update parity information.

Additional RAID levels

- **RAID 6**: block-level striping with double distributed parity (can tolerate two disk failures)
- **RAID 10**: a combination of RAID 1 and RAID 0 (striping + mirroring)
- Many other levels and variations exist, each with different trade-offs in terms of performance, redundancy, and storage efficiency
 - Levels have become marketing tools, higher levels are typically some combination (often nested) of those we've talked about
 - e.g., **RAID 50** (striping across multiple RAID 5 sets), **RAID 60** (striping across multiple RAID 6 sets), etc.
- **Hot-spares**: a drive in a RAID group that is blank and inactive. Can be used to immediately rebuild a RAID should a single drive fail without waiting for humans

Choosing a RAID level

- Depends on your needs and constraints:
 - Performance requirements (read vs write, small vs large)
 - Redundancy needs (tolerance for disk failures)
 - Storage efficiency (how much usable space do you need?)
 - Budget (cost of additional disks for redundancy)
- For example:
 - RAID 0 for high performance with no redundancy (e.g., gaming)
 - RAID 1 for critical data that requires high redundancy (e.g., financial records)
 - RAID 5 for a balance of performance and redundancy (e.g., general-purpose servers where a long rebuild time is acceptable)
 - RAID 6 for environments where data integrity is paramount and multiple disk failures are a concern (e.g., archival storage)

What isn't clear?

Comments? Thoughts?

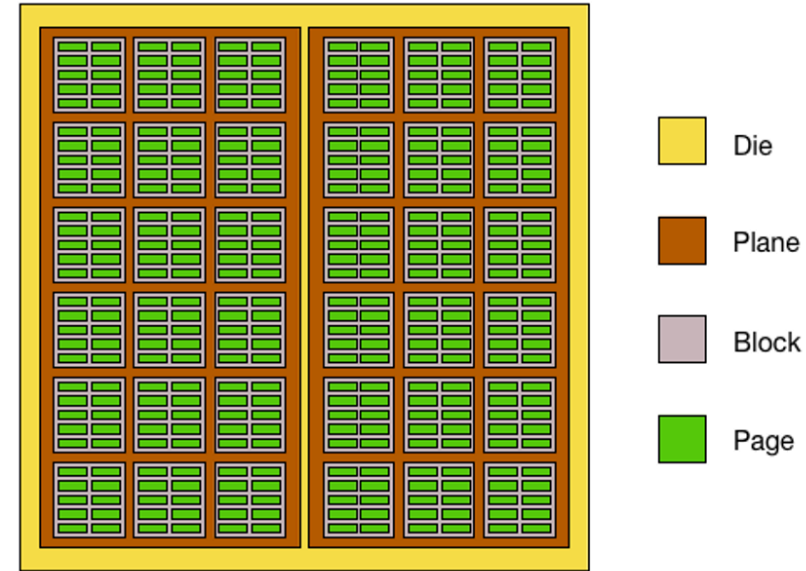
SSDs

Questions to consider

- What are the three levels of NAND flash organization, and what is the smallest unit of read/write vs. erase?
- Why can't an SSD overwrite a page in place, and what sequence of operations is required to update data?
- What is write amplification, and why does it occur in NAND flash?
- What is wear leveling, and why is it necessary for SSD longevity?
- What does the Flash Translation Layer (FTL) do, and why does the OS need it?

SSDs: NAND Flash Basics

- SSDs use NAND flash memory, which is organized into planes, blocks, and pages
 - Plane: a collection of blocks that can be accessed in parallel
 - Block: a collection of pages (e.g., 128-256 pages per block)
 - Page: the smallest unit of read/write (e.g., 4KB-16KB)



NAND rules

- Cells store 1–4 bits (SLC → QLC); more bits per cell = cheaper, slower, fewer write cycles
- Always **read** an entire page (e.g., 4KB-16KB)
- Always **write** an entire page (to change a single byte, you must write the whole page)
 - **Fast(er)**: 100s of microseconds
- Pages cannot be overwritten in place — page can only be written once after an erase
 - An update requires: read page into memory → modify → write to a new location → mark old page invalid
- **Erase** is done at the block level (e.g., 128–256 pages); all pages erased together, every bit reset to 1
 - All data must be copied out safely before erase
 - **Slow**: few milliseconds

SSDs: Performance characteristics

Device	Read (μs)	Program (μs)	Erase (μs)
SLC	25	200-300	1500-2000
MLC	50	600-900	~3000
TLC	~75	~900-1350	~4500

SSD challenges

- Flash memory is written in pages, erased in blocks
 - **Write amplification**: due to the need to erase blocks before writing, the actual amount of data written to the flash can be much larger than the amount of data intended to be written by the host
 - e.g., a single byte update can trigger a read-modify-write cycle for an entire block
 - **Garbage collection**: the process of reclaiming space from invalid pages, which can impact performance if not managed properly
- Flash memory wears out after a certain number of program/erase cycles, leading to potential data loss and reduced lifespan
 - **Wear leveling**: to prevent premature failure of flash cells, data must be distributed evenly across the flash memory

SSDs: Flash Translation Layer (FTL)

- The OS cannot deal with all of this complexity. The **FTL** is a layer of software/firmware that manages everything related to flash memory, presenting a simple block device interface to the OS
- Responsibilities of the FTL:
 - Address mapping: translating LBAs to PBAs
 - Wear leveling: ensuring even distribution of writes across the flash memory
 - Garbage collection: reclaiming space from invalid pages
 - Bad block management: handling blocks that have worn out or are defective
- The FTL is critical for the performance and longevity of an SSD, as it abstracts away the complexities of flash memory management from the host system

What isn't clear?

Comments? Thoughts?